# PROJECT REPORT: PROJECT BERMUDA

# COURsE: FUnDAMEnTAls In ARTIFICIAl InTEllIgEnCE AnD MAChInE lEARnIng

---

**1. Project Objective**

The objective of this project is to develop a content-based music recommendation system that suggests similar songs to a user based on a given song title. By leveraging Natural Language Processing (NLP) techniques, the system analyzes the textual metadata of songs such as genre, artist, and track name to compute a similarity score between tracks. The AI then returns a ranked list of the most similar songs, providing a utility-based recommendation that does not require user history or collaborative filtering data.

**2. Syllabus Mapping & Application**

This project was specifically architected to demonstrate core concepts from the Introduction to AI syllabus, primarily focusing on the following:

- **Knowledge Representation & Data Preprocessing:** The raw data, represented in a CSV file, is transformed into a structured format suitable for machine learning. This involves feature engineering, where textual information from multiple columns (genre, artist_name, track_name) is combined into a single, rich feature set. This step converts unstructured data into a format that an AI model can understand and process.
- **Machine Learning & Vectorization:** The core AI technique used is **Vectorization and Similarity Measurement**. The TfidfVectorizer (Term Frequency-Inverse Document Frequency) from scikit-learn is applied to the combined text features. This algorithm transforms text data into a numerical matrix (TF-IDF matrix) where each row represents a song and each column represents a unique word, weighted by its importance in the document (song) relative to the entire corpus.
- **Uninformed Search & Optimization:** The system implements a utility-based search to find the "nearest neighbors" to a given song. After the TF-IDF matrix is created, the cosine_similarity function is used as a mathematical measure to find the most similar items. This process is analogous to a search in a high-dimensional vector space, where the goal is to find the points (songs) closest to a query point, optimizing for maximum similarity (i.e., minimum distance in vector space).
- **Exploratory Data Analysis (EDA):** The project begins with an EDA phase, using libraries like matplotlib and seaborn to visualize the distribution of songs by genre and by artist. This step is crucial for understanding the underlying dataset and validating the quality of the data before building the recommendation engine.

**3. System Architecture**

The project follows a modular architecture, with a clear separation of data handling, model building, and user interaction.

- **Data Ingestion & EDA (Python):** The process begins with loading the dataset (tcc_ceds_music.csv) into a Pandas DataFrame. This module is responsible for data cleaning (handling null values), exploratory analysis (creating visualizations), and initial feature engineering by combining the relevant textual columns into a single combined_features field.
- **Modeling & Vectorization (scikit-learn):** This is the core of the AI engine. It takes the preprocessed data and uses the TfidfVectorizer to convert the textual features into a numerical TF-IDF matrix.
  The cosine_similarity function then calculates a pairwise similarity matrix between all songs in the dataset.
- **Recommendation Engine (Python):** The recommendation function get_recommendations() acts as the query handler. It takes a user-provided song title, looks up its index in the dataset, retrieves its similarity scores from the pre-computed matrix, sorts the scores in descending order, and returns the top n most similar songs, excluding the query song itself.
- **Visualization & Output (matplotlib):** The final results are not just printed but also visualized using a bar chart. This provides an intuitive, graphical representation of the recommendations, showing the song title, artist, and genre in a user-friendly manner.

- **4. Implementation Details**

The implementation follows a step-by-step process to build the recommendation engine:

1. **Data Loading:** The dataset is loaded from a CSV file (tcc_ceds_music.csv) uploaded to the Colab environment.
2. **Exploratory Data Analysis (EDA):**
   - o A count plot is generated to visualize the top 10 most frequent genres in the dataset.
   - o A bar plot is created to show the top 10 artists with the highest number of songs.
   - o This analysis helps in understanding the data's composition and potential biases.
3. **Preprocessing:**
   - o A new column combined_features is created by concatenating the genre, artist_name, and track_name for each song. This creates a rich text document for every song.
4. **Vectorization:**

- The TfidfVectorizer is initialized with stop_words='english' to filter out common, non-informative words. The vectorizer is then fitted to the combined_features and transforms it into a TF-IDF matrix. This matrix represents the importance of each word to each song.

5. **Similarity Calculation:**
   - The cosine_similarity function computes the cosine of the angle between every pair of songs in the high-dimensional vector space. The result is a square similarity matrix. A value close to 1 indicates high similarity.
6. **Recommendation Function:**
   - The function get_recommendations(song_title, data, cosine_sim, top_n=10) is defined.
   - It first checks if the given song_title exists in the dataset.
   - It retrieves the index of the input song and then uses the pre-computed similarity matrix to get similarity scores for all other songs.
   - These scores are sorted in descending order, and the top n songs (excluding the input song) are returned as a DataFrame.
7. **Testing & Visualization:**
   - The system is tested with the song 'hum tujhse mohabbat kar ke'.
   - The recommendations are printed in a tabular format.
   - A bar chart is generated to visually compare the recommended songs by their associated artists, providing a clear, graphical summary of the results.

**5. Critical Reflection & Future Directions**

**Algorithmic Analysis:**

The current implementation, based on TF-IDF and Cosine Similarity, is a classic and highly effective content-based filtering approach. It is **interpretable** (similarity is based on shared terms) and does not suffer from the "cold start" problem common in collaborative filtering systems, as it only requires song metadata to make recommendations. The system's performance is efficient for a dataset of this size (~28,000 songs), with the similarity matrix being pre-computed offline, allowing for fast real-time queries.

However, the approach has notable limitations:

- **Limited Feature Set:** The system only uses genre, artist, and track name. It does not analyze deeper features like musical attributes (tempo, key, loudness), lyrics, or user listening history. This means it might recommend songs that are textually similar but sonically very different.
- **Static Data:** The model is static and does not adapt to user feedback. It cannot learn from a user's preferences over time to provide more personalized recommendations.

**Future Directions:**

To enhance the system, several advanced techniques can be implemented:

- **Advanced Feature Engineering:** Incorporating more features, such as song lyrics (using NLP on lyrics text), audio features from an API like Spotify's Web API, or metadata like release year, could provide a more nuanced understanding of a song's identity.

- **Hybrid RecommendationSystems:** The system could be expanded toahybridmodel that combines content-based filtering (thisproject) with collaborative filtering. Collaborativefiltering would analyze patterns from many users'listening habits to suggest songs that similarusersenjoy, significantly improving personalization.
- **Deep Learning Integration:**Amore sophisticated approach would be tousedeeplearning models. For example, a neural networkcould be trained to create high-quality "embeddings"(vector representations) for songsbylearning from both textual data and audiosignals.These embeddings could then be used for evenmore accurate similarity searches.
- **Real-time Feedback Loop:**Integrating a feedback mechanism (e.g., a "like" or "dislike" button) could allow the model to dynamically adjust recommendations for a user, creating a personalized and interactive experience.

**6. Conclusion** The Music Recommendation System successfully demonstrates a practical application of core AI concepts, including data preprocessing, knowledge representation through vectorization, and algorithmic search for finding nearest neighbors. By utilizing TF-IDF and cosine similarity, the system provides a robust and efficient method for content-based music discovery. While the current implementation serves as a strong foundational model, the project successfully showcases the power of machine learning in building intelligent systems that can analyze and derive meaningful insights from data to enhance user experience.

**MADE & AUTHORED BY :- SHUBH TIWARI (25BAI10799)**